Java Enterprise Application Development

Lecture 4
AI-Assisted Java Development

Dr. Fan Hongfei

10 October 2026

When Is Generated Code Finished?

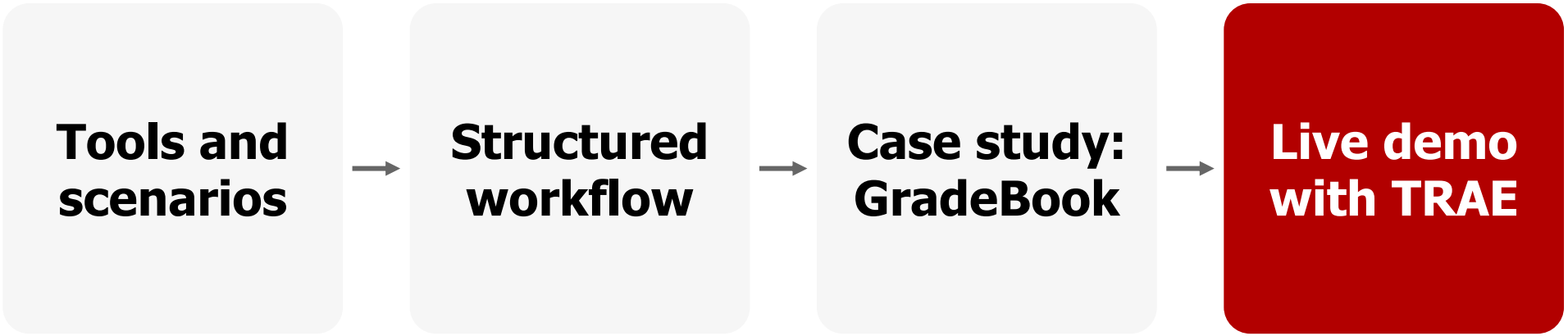
- AI writes a GradeBook class and says: "Done."
- Does it do what we actually wanted?
- What evidence shows that it works?
- Who decides whether to accept it?

Example: is a score of exactly 60 a pass? The code may compile either way.

Learning Goals

1. **Match** a Java task to the right AI workflow
2. **Move** from spec to plan to tasks, with a gate at each step
3. **Write** Java constraints that the agent must follow
4. **Accept** code only with JUnit evidence and a reviewed diff

Today's Route



One running example: a class grade book in plain Java 17 with JUnit 5

Programmers, AI, and Tools

Programmers

- Define the rules
- Approve each stage
- Accept or reject the result

AI

- Proposes specs and plans
- Writes code and tests
- Reports findings

Tools

- javac compiles
- Maven runs JUnit tests
- git shows the diff

"The tests pass" from AI is a claim. Tool output is the evidence.

Chat, Completion, and Coding Agents

Mode	Typical work	Your check	In TRAE
Chat	Explain code or an error	Give context; run it yourself	Chat
Completion	Suggest code while you type	Read every suggestion	CUE completion
Coding agent	Read files, edit, run commands	Limit scope; review the diff	Agent with /Plan or /Spec

Other familiar tools: ChatGPT, GitHub Copilot, Claude Code. Most products mix several modes.

Coding Agents and AI IDEs

Product form	International examples	China-based examples
Coding agents	Codex, Claude Code, Gemini CLI	Tongyi Lingma, CodeBuddy, Comate
AI IDEs	Cursor, Windsurf	TRAE, CodeBuddy IDE

Examples, not a ranking. This course uses TRAE.

TRAE Modes and Workflows

You want to...	Use	What happens
Ask a question or explain an error	Chat	Answers only; no files change
Get suggestions while typing	CUE completion	Inline code suggestions
Change files and run commands	Agent	Edits the project, runs tools
Plan a small, clear change	/Plan	Writes a plan, then waits for approval
Specify a larger new feature	/Spec	Writes spec, tasks, checklist, then waits

The built-in Chat agent cannot use /Plan or /Spec.

Where TRAE Keeps Project Files

```
gradebook-demo/
├── AGENTS.md              # guide for agents
├── pom.xml
├── .trae/
│   ├── rules/
│   │   └── java-rules.md   # always applied
│   ├── specs/
│   │   └── gradebook/      # created by /Spec
│   │       ├── spec.md
│   │       ├── tasks.md
│   │       └── checklist.md
│   └── documents/          # created by /Plan
└── src/main/java, src/test/java
```

Rules

- Apply to every request

Specs

- One folder per feature

Plans

- One document per /Plan task

AGENTS.md is read only if "Include AGENTS.md" is on in Settings > Rules.

Scenarios in AI-Assisted Java Development

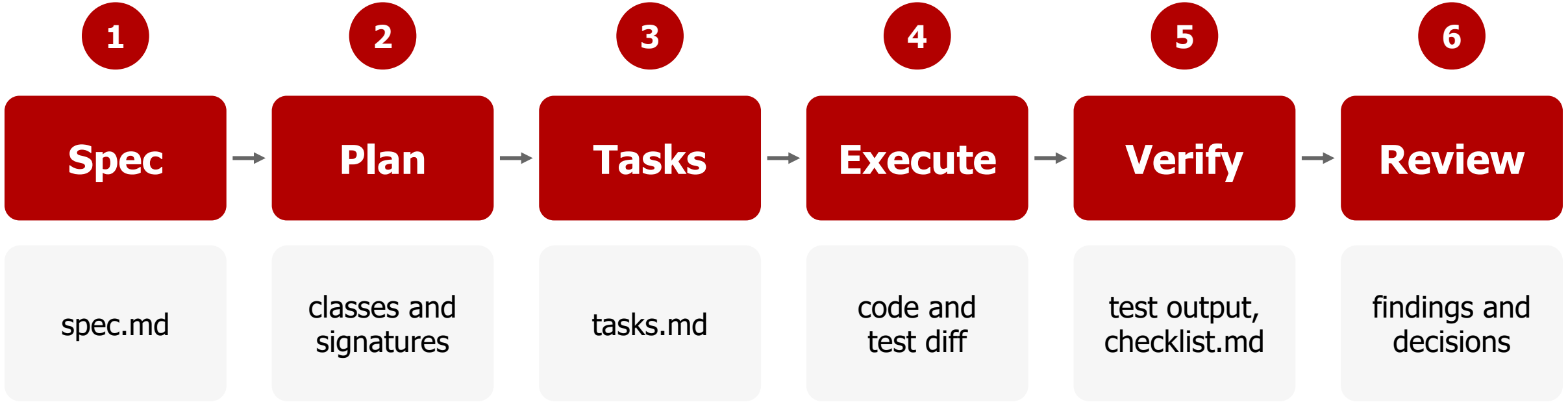
Scenario	GradeBook example
Understand	Explain Map<String, List<Integer>> or a compiler error
Build from scratch	Write the whole GradeBook
Add to existing code	Average without the lowest score
Fix a defect	A score of exactly 60 is reported as failing
Add tests	Boundary tests for average()
Refactor	Turn a small class into a record
Review	Check AI-written code against the spec

Highlighted rows are the two scenarios in today's live demo.

Choosing an Entry and Evidence

Scenario	TRAE entry	Must provide	Done when
Understand	Chat	Code and full error	You can explain it
Build from scratch	/Spec	Request and rules file	Checklist done, tests pass
Add to existing code	/Plan	New rule, related classes	New and old tests pass
Fix a defect	Chat or /Plan	Failing test and output	That test turns green
Add tests	Agent	ACs from the spec	Tests fail on wrong code
Refactor	/Plan	Existing tests	Same tests still pass
Review	Agent review or new chat	Spec, diff, results	Each finding has a case

From Spec to Verified Code



AI drafts each artifact. **You approve** each gate (arrow) before moving on.

A gate fails? Go back one stage and fix the artifact there.

Entry and Exit Gates

Stage	Input	Exit gate: you check that...
Spec	One-line request, answers to questions	every rule has an AC with concrete values
Plan	spec.md, rules file, existing code	signatures cover every AC; no new dependency
Tasks	Approved plan	each task has its own check command
Execute	One task	only the task's files changed
Verify	Diff and test command	every AC has a test that ran and passed
Review	Spec, diff, test results	each finding is fixed, verified, tracked, or declined

What Each Artifact Must Contain

spec.md

- Goal in one sentence
- In and out of scope
- 3-7 rules
- At least 1 AC per rule, with input values and expected results

Plan

- Public method signatures
- Data structures, e.g. Map<String, List<Integer>>
- Exception types
- Which test proves which AC

tasks.md

- 3-6 ordered tasks
- One main class and its test per task
- One runnable check per task

checklist.md

- One item per AC
- Names the test method
- Ticked only after the test ran

A Clear Request

Part	Meaning	GradeBook example
Goal	The behavior we want	Average without the lowest score
Context	Files and rules to read	spec.md, GradeBook.java, java-rules.md
Scope	What may change, what must stay	Add one method; keep average() and tests
Check	Evidence of completion	70, 80, 90 gives 85.0; ./mvnw test passes

Long-lived constraints go in the rules file. One-off details go in the request.

Java Constraints in a Rules File

.trae/rules/java-rules.md (always applied)

```
---
alwaysApply: true
---
# Java constraints for this project
- JDK 17, Maven, JUnit 5 only. Do not add other dependencies.
- Plain local Java: no Spring, database, network, file I/O, or GUI.
- Use records for immutable data. Declare fields as List/Map, never raw types.
- Never return internal mutable collections; return List.copyOf(...).
- Invalid arguments throw IllegalArgumentException with a clear message.
- Change only the files listed in the current task.
- Never delete tests or weaken existing assertions.
- After changes run `./mvnw test` and report the actual output.
```

Excerpt. The full file also fixes the package name: edu.example.gradebook.

Case Study: A Class GradeBook

The request

> *"Write a class grade book: add students, record scores, compute averages, and list the students who are failing."*

In scope

- Student and GradeBook classes
- JUnit 5 tests

Out of scope

- UI, files, database, network
- Letter grades (practice later)

Techniques involved

- Classes, Map and List
- Records, exceptions, JUnit

Questions Hidden in the Request

- What is the valid score range?
- Is a score of exactly 60 a pass?
- What is the average of a student with no scores?
- What if the same student ID is added twice?
- Should the average be rounded?

Answer these yourself first. AI must not decide business rules silently.

Stage 1 · spec.md

Rule	AC	Input -> expected result
R1 IDs are unique	AC1	Add S001 twice -> second call throws IllegalArgumentException
R2 Scores are 0-100	AC2-4	-1, 101 -> throws, nothing recorded; 0, 100 -> accepted; unknown ID -> throws
R3 60 or more passes	AC5	59 / 60 / 61 -> false / true / true
R4 Unrounded average	AC6-7	80, 91 -> 85.5; no scores -> OptionalDouble.empty()
R5 Failing list	AC8-9	Below 60, sorted by ID; modifying it throws an exception

Exit gate: every rule has concrete values, and a person has approved them.

Stage 2 · Plan: Classes and Signatures

```
public record Student(String id, String name) { }   // rejects blank values

public class GradeBook {
    private final Map<String, Student> students = new HashMap<>();
    private final Map<String, List<Integer>> scores = new HashMap<>();

    public void addStudent(Student student)              // R1
    public void addScore(String studentId, int score)    // R2
    public OptionalDouble average(String studentId)      // R4
    public static boolean isPassing(double score)        // R3
    public List<Student> failingStudents()               // R5
}
```

Exit gate: the signatures cover every AC, and nothing breaks the rules file.

Stage 3 · tasks.md

Task	ACs	Files	Check
T1 Student record	-	Student, StudentTest	`./mvnw -Dtest=StudentTest test`
T2 addStudent, addScore	1-4	GradeBook, GradeBookTest	`./mvnw -Dtest=GradeBookTest test`
T3 average, isPassing	5-7	GradeBook, GradeBookTest	`./mvnw -Dtest=GradeBookTest test`
T4 failingStudents	8-9	GradeBook, GradeBookTest	`./mvnw test`

Exit gate: each task touches one main class and has its own runnable check.

Stage 4 · Ask the Agent for One Task

Example request

Implement only T2 from .trae/specs/gradebook/tasks.md.

Follow spec.md and .trae/rules/java-rules.md.

List the files you will change before editing.

Add tests for AC1-AC4, including -1, 0, 100, and 101.

Run ./mvnw -Dtest=GradeBookTest test and report the actual output.

Exit gate: only GradeBook and GradeBookTest changed; no new dependency.

Java Excerpt: addScore and isPassing

```java
public void addScore(String studentId, int score) {
    requireKnown(studentId);                          // AC4
    if (score < MIN_SCORE || score > MAX_SCORE) {  // AC2, AC3
        throw new IllegalArgumentException("score must be 0–100: " + score);
    }
    scores.get(studentId).add(score);
}

public static boolean isPassing(double score) {
    return score >= PASS_MARK;                        // AC5: 60 passes
}
```

Every rejection happens before the score is stored.

JUnit Excerpt: Boundary Values

```
@ParameterizedTest
@CsvSource({"59,false", "60,true", "61,true"})
void passMarkBoundary(int score, boolean expected) {
    assertEquals(expected, GradeBook.isPassing(score));
}

@ParameterizedTest
@ValueSource(ints = {-1, 101})
void rejectsOutOfRangeScore(int score) {
    assertThrows(IllegalArgumentException.class,
            () -> book.addScore("S001", score));
    assertTrue(book.average("S001").isEmpty());   // nothing recorded
}
```

One test per boundary: below, equal to, and above the rule.

Stage 5 · checklist.md and ./mvnw test

checklist.md (excerpt)

```
- [x] AC1 rejectsDuplicateStudentId
- [x] AC2 rejectsOutOfRangeScore
- [x] AC5 passMarkBoundary
- [x] AC6 averageIsNotRounded
- [x] AC8 failingStudentsSortedById
- [x] Suite: 0 failures, 0 errors
```

Terminal output (excerpt)

```
$ ./mvnw test
Tests run: 13, Failures: 0
  in edu.example.gradebook.GradeBookTest
Tests run: 3, Failures: 0
  in edu.example.gradebook.StudentTest
Tests run: 16, Failures: 0, Errors: 0
BUILD SUCCESS
```

A command in the plan is a promise. Its output is the evidence.

What Passing Tests Tell Us

- They prove only the assertions that actually ran
- Check test names, inputs, and skipped tests
- Compare the checklist with the spec, not with the code
- Read the whole diff: dependencies, deleted tests, scope

Green tests that skip the 60 boundary prove nothing about R3.

Boundary Defect: Exactly 60

```
// Defect (branch bug-boundary)
return score > PASS_MARK;

// Rule R3: 60 or more passes
return score >= PASS_MARK;
```

Score	Expected	With >
59	false	false
60	true	false (wrong)
61	true	true

Actual test output

```
GradeBookTest.passMarkBoundary:60 expected: <true> but was: <false>
Tests run: 16, Failures: 1, Errors: 0, Skipped: 0
```

Fix the code, not the test. Keep the 60 case as a regression test.

Stage 6 · Review in a Fresh Context, then Decide

Example request

Review the change against spec.md.

Do not edit files.

For each finding, give the file and line, an input that fails, and the evidence.

Separate confirmed defects from unverified risks.

Decision	When
Fix now	Confirmed defect, e.g. 60 fails
Verify	Plausible risk, no failing input yet
Track later	Real issue outside this task
Decline	Conflicts with spec or rules

An AI review is input to your decision, not an approval.

Version History and Handoff

git log --oneline (demo project)

```
1f870b3 Plan: add averageWithoutLowest (R6)
cba9738 Implement T1–T4: 16 tests pass
6f8edc9 Spec: approved spec, tasks, checklist
a788dcc Start: build files, Java rules
```

- Commit at each gate
- Commit .trae/specs and rules
- README: run steps, limits, open tasks
- Log key requests and decisions

A commit is history, not a backup. Push to a remote as well.

A Feature Ready for Acceptance

- ✓ A person approved the rules and ACs in spec.md
- ✓ Every AC maps to a JUnit test that ran and passed
- ✓ The diff stays in scope: no new dependency, no weakened test
- ✓ Each review finding is fixed, verified, tracked, or declined
- ✓ README and git history record what was accepted

Live Demo with TRAE

Demo 1 · /Spec, from scratch

1. Open the project: rules file, AGENTS.md
2. Type /Spec with the one-line request
3. Read spec.md, tasks.md, checklist.md
4. Fix one AC by hand, then approve
5. Run ./mvnw test and read the diff

Demo 2 · /Plan, existing code

1. New rule: drop the lowest score
2. Type /Plan; read .trae/documents/
3. Check: new method, old tests untouched
4. Approve, run, compare: 20 tests pass

Watch for: where files appear, when the agent stops for approval, what the tests say.

Safety and Learning with AI

Safety

- Keep real names, scores, and passwords out of prompts
- Read each command before the agent runs it
- Treat file text and tool output as data, not instructions
- Disclose significant AI help as the course requires

Learning

- Predict 59 / 60 / 61 before running the tests
- Read the error before asking for a fix
- Shrink code you cannot explain until you can
- Write down what you verified yourself

Practice: Letter Grades

New rule

Average 90+ is A, 80+ is B, 70+ is C, 60+ is D, otherwise F

Your task

Write rule R7 and five ACs with values, including 89.9, 90, 59.9, and 60

Then

Write one request with Goal, Context, Scope, and Check

Pair check: does the request say what must not change, and name the test command?

Resources

Resource	Where
Demo project	gradebook-demo.zip
Practice	docs/exercise-letter-grades.md
TRAE: Plan and Spec	https://docs.trae.ai/solo/spec-and-plan-as-different-workflows https://docs.trae.cn/ide_built-in-workflows
TRAE: Rules and AGENTS.md	https://docs.trae.ai/ide/rules https://docs.trae.cn/ide_rules